Resumen de Clase: Desarrollo Guiado por Pruebas (TDD)

Materia: Ingeniería y Calidad de Software (ICSW)

Clase: *Desarrollo Guiado por Pruebas (TDD) — Partes 1 y 2* | ***Fecha:*** *06/10/2026* | ***Fuente:*** *Apuntes de clase*

ÍNDICE DE CONTENIDOS Y TEMAS DE LA CLASE

1	1. Introducción y Filosofía de Desarrollo Guiado por Pruebas (TDD)
2	2. El Ciclo Iterativo TDD: Red, Green, Refactor
3	3. Las Tres Leyes de TDD y Patrones de Pruebas Unitarias
4	4. Ejemplo Práctico Integrado: Sistema TaxiMobile
5	5. Pautas Evaluativas, Cobertura (+20 Tests) y Mocks/Stubs

---

1. Introducción y Filosofía de Desarrollo Guiado por Pruebas (TDD)

El **Desarrollo Guiado por Pruebas** (*Test Driven Development - TDD*) es una técnica de ingeniería de software en la cual el desarrollo del código de producción es guiado de manera directa por la definición previa de pruebas automatizadas. En lugar del esquema tradicional de escribir el código de producción y probarlo posteriormente, TDD invierte completamente el proceso: primero se diseña y escribe la prueba unitaria que falla, y luego se construye únicamente el código necesario para satisfacer dicha prueba.

Un Cambio de Paradigma Mental

Adoptar TDD no se limita a utilizar un framework de pruebas, sino que requiere internalizar un cambio cultural en la forma de programar. El desarrollador debe sentarse primero a pensar en las especificaciones del problema: qué entradas recibirá el componente, qué comportamientos se esperan y cuáles son las condiciones de error o borde. Diseñar las pruebas antes de programar fuerza una comprensión profunda de los requerimientos y reglas del negocio.

Evolución de Técnicas y Referentes

TDD fue popularizado por **Kent Beck** (considerado el padre de esta técnica). En entornos ágiles modernos coexiste con otras metodologías derivadas como **ATDD** (*Acceptance Test-Driven Development*), enfocada en pruebas de aceptación desde el punto de vista del usuario, y **Spec-Driven Development** (desarrollo guiado por especificaciones).

Beneficios e Impacto en la Calidad del Software

- **Código más robusto y seguro:** Cualquier modificación futura o mantenimiento que rompa el comportamiento existente es detectado de forma inmediata al reejecutar la batería de pruebas automatizadas.
- **Cobertura total de pruebas (100% Coverage):** Dado que no se escribe código de producción sin un test que lo justifique, por definición se elimina el código no probado o caminos no verificados.

- **Alta cohesión y bajo acoplamiento:** Probar unidades pequeñas y aisladas obliga a diseñar clases y métodos focalizados, con responsabilidades únicas e interfaces limpias.
- **Diseño e integración con APIs externas:** Pensar tempranamente cómo se va a consumir y probar una API o servicio externo fuerza a definir contratos de comunicación y manejo de errores antes de conectar los componentes reales.
- **Criterio objetivo de 'Terminado' (Done):** La funcionalidad se considera terminada cuando todas las pruebas unitarias diseñadas corren y pasan exitosamente en verde.

> **Concepto Clave de Calidad TDD**
> TDD es principalmente una técnica de diseño más que una simple estrategia de testing. Al obligar a pensar cómo se consumirá y verificará un método antes de construirlo, garantiza que el software sea naturalmente modular, desacoplado y fácil de mantener a lo largo del tiempo.

2. El Ciclo Iterativo TDD: Red, Green, Refactor

El núcleo operativo de TDD transcurre a través de un ciclo iterativo de tres pasos estrictos y continuos, conocido como el flujo *RED - GREEN - REFACTOR*.

Las Tres Fases del Ciclo

1. Fase RED (Rojo): Se escribe una prueba unitaria basada en el requerimiento o criterio de aceptación. Como el código de producción aún no ha sido implementado, la prueba DEBE fallar al ejecutarse (incluso el hecho de que no compile o no encuentre la clase/método cuenta como un fallo válido). Esto confirma que la prueba está midiendo algo que realmente falta por construir.

2. Fase GREEN (Verde): Se escribe la menor cantidad de código de producción posible para lograr que la prueba pase. En esta fase no importa la elegancia ni la optimización de la solución; incluso se permite hardcodear valores de retorno iniciales con tal de satisfacer el assert y poner la prueba en verde de forma rápida.

3. Fase REFACTOR (Refactorización): Con la seguridad de tener la prueba en verde, el desarrollador analiza el código recién escrito para mejorar su estructura interna. Se elimina la duplicación de código, se extraen métodos o clases, se renombran variables y se refina el diseño sin alterar el comportamiento externo. Tras cada refactorización, las pruebas se ejecutan nuevamente para asegurar que todo continúe funcionando.

Refactorización vs. Rediseño

El refactor es un proceso continuo que no debe confundirse con reescribir el sistema. Consiste en pequeños cambios incrementales para mantener el código limpio (*Clean Code*) y erradicar la deuda técnica. Detectar 'código que huele mal' (*Code Smells*) —como métodos demasiado extensos, duplicación de lógica o demasiados parámetros— es la señal para iniciar la fase de refactorización.

> **El Flujo Continuo de Refactorización**
> Las pruebas automatizadas actúan como una red de seguridad inestimable. Cuando refactorizamos para eliminar duplicaciones o simplificar la lógica, si cometemos un error, la suite de pruebas fallará inmediatamente en rojo, señalando el punto exacto a corregir.

3. Las Tres Leyes de TDD y Patrones de Pruebas Unitarias

Para aplicar TDD con rigurosidad disciplinaria, **Robert C. Martin** (*'Uncle Bob'*) formuló las Tres Leyes que rigen la construcción del software:

Las Tres Leyes de TDD (Uncle Bob)

Primera Ley: No escribirás código de producción sin antes haber escrito una prueba unitaria que falle.

Segunda Ley: No escribirás más de una prueba unitaria de la que sea suficiente para fallar (y no compilar es fallar).

Tercera Ley: No escribirás más código de producción que el estrictamente necesario para hacer pasar la prueba unitaria que actualmente falla.

Patrones y Buenas Prácticas de Redacción de Pruebas

- **Pruebas cortas y cohesivas:** Cada prueba unitaria debe verificar un único aspecto o escenario puntual. Si un test prueba múltiples condiciones a la vez, debe dividirse en tests independientes.
- **Nombres descriptivos y estandarizados:** El nombre del test debe explicar qué escenario está probando (ej. *test_mostrar_taxis_libres_en_barrio_nueva_cordoba*). El prefijo test_ ayuda a los frameworks a reconocer los métodos automáticamente.
- **Aislamiento e Independencia:** Ningún test debe depender del resultado o del estado dejado por la ejecución de un test anterior. Cada prueba se configura y limpia de forma independiente.

Patrón de Estructuración AAA (Arrange - Act - Assert)

Arrange (Precondiciones): Preparar el entorno, instanciar objetos, configurar permisos de usuarios y simular datos ficticios de prueba.

Act (Ejecución): Invocar el método o funcionalidad exacta bajo análisis.

Assert (Verificación): Comprobar mediante afirmaciones que los resultados coinciden exactamente con la especificación requerida.

> **Estructura Limpia AAA**
> Aplicar de forma consistente el patrón Arrange-Act-Assert facilita la lectura técnica y auditoría del código de test, permitiendo a cualquier desarrollador comprender el escenario probado en segundos.

4. Ejemplo Práctico Integrado: Sistema TaxiMobile

Durante la clase práctica se analizó un ejercicio completo de aplicación TDD sobre el sistema **TaxiMobile** (plataforma para la gestión centralizada de pedidos de taxis).

Requerimiento (User Story y Criterios)

Historia de Usuario: *'Como administrador de la central quiero ver la ubicación de todos los taxis de la central si tienen viajes en curso para saber la disponibilidad.'*

Criterios de Aceptación: Se muestran taxis en todos los estados (verde = libre, ocupado, solicitado); deben tener activado el sistema GPS de ubicación; se permite filtrar por barrio, por estado y por chapa del taxi.

Paso 1: Configurar Precondiciones (Arrange)

Antes de llamar a la función a probar, se configura el usuario con rol de Administrador de la Central, se inicia sesión, se define la ciudad de Córdoba y se cargan taxis de prueba ficticios con coordenadas GPS activas:

```
def test_mostrar_taxis_libres_en_barrio_nueva_cordoba():
        # 1. PRECONDICIONES (Setup / Arrange)
        usuario = crear_usuario_administrador_central()
        iniciar_sesion(usuario)
        configurar_sistema_ciudad('Cordoba')

        # Cargar taxi de prueba con GPS activo
        crear_taxi(chapa='ABC1', estado='libre', barrio='Nueva Cordoba', gps_activo=True,
lat=-31.42, lon=-64.18)
```

Paso 2: Ejecución de la Funcionalidad (Act)

Se simula la interacción del usuario seleccionando la opción 'Ver mapa de taxis' e invocando la función descubierta mostrar_mapa:

```
# 2. EJECUCION (Act)
        opciones = seleccionar_opcion('ver_mapa_taxis')
        mapa_mostrado = mostrar_mapa(opciones, estado='libre', barrio='Nueva Cordoba')
```

Paso 3: Verificación con Afirmaciones (Assert)

Se valida formalmente que los resultados devueltos coincidan de forma exacta con lo requerido por el negocio:

```
# 3. VERIFICACION (Assert)
        assert mapa_mostrado.ciudad == 'Cordoba', 'Error: El mapa no pertenece a Cordoba'
        assert all(taxi.estado == 'libre' for taxi in mapa_mostrado.taxis), 'Error: Hay taxis
no libres'
        assert all(taxi.icono == 'verde' for taxi in mapa_mostrado.taxis), 'Error: El icono no
es verde'

        print('SUCCESS: Test de visualizar taxis libres en barrio paso correctamente')
```

Paso a Verde y Refactorización

Al ejecutar este test por primera vez, fallará (fase RED) porque la función mostrar_mapa no existe. En la fase GREEN, se construye el método retornando una estructura básica (JSON/diccionario) con los campos esperados (ciudad, taxis, estado, icono). Finalmente, en la fase REFACTOR se limpia el código, se parametriza la búsqueda y se asegura la calidad del componente.

> **Caso TaxiMobile: Verificación de Estados**
> El test valida que únicamente los taxis en estado 'Libre' aparezcan reflejados en el mapa con su correspondiente marcador verde, garantizando que los taxis ocupados o fuera de servicio no saturen la vista del usuario.

5. Pautas Evaluativas, Cobertura (+20 Tests) y Mocks/Stubs

En la segunda parte de la clase práctica se repasaron las condiciones y pautas obligatorias para la entrega del Trabajo Práctico Evaluable de TDD en la materia.

Exigencias de Interfaz Gráfica y Entorno Visual

A pesar de que las ejecuciones automatizadas de los tests unitarios se lanzan por consola o mediante un runner de pruebas (ej. PyUnit/unittest), la aplicación final debe contar con una **interfaz gráfica visual** (aplicación web responsiva recomendada o mobile). Está estrictamente prohibido presentar la aplicación interactiva por consola.

Volumen de Pruebas y Cobertura Exigida (+20 Tests)

- **Cantidad mínima de pruebas:** Cada trabajo práctico debe contar con una batería amplia de **más de 20 pruebas unitarias** registradas y agrupadas en suites de test.
- **Pruebas de Camino Feliz (Positivas):** Verificar comportamientos correctos con entradas válidas (ej. filtrados por barrio, rangos de fechas permitidos).
- **Pruebas de Fallo / Casos Borde (Negativas):** Diseñar escenarios donde se intente forzar una violación de regla de negocio o ingresar datos inválidos (ej. fechas futuras, montos vacíos, usuarios sin permisos) esperando que el sistema bloquee la acción y notifique el error correspondiente.

Uso de Mocks, Stubs y Test Doubles

Cuando el sistema debe interactuar con servicios externos (ej. cobro con tarjeta de crédito, validación con pasarelas de pago, lectura de dispositivos GPS en tiempo real), es necesario utilizar dobles de prueba (*Mocks* o *Stubs*). Estos objetos simulados permiten probar la lógica del sistema de forma aislada sin depender de la conectividad o disponibilidad del servicio externo.

Rol de los Profesores como Product Owners

Durante el desarrollo del trabajo práctico, los docentes asumen el rol de *Product Owners*. Los estudiantes disponen de instancias de consulta dedicadas para presentar sus listas de pruebas y aclarar dudas sobre los requerimientos de negocio antes de la entrega final.

> **Pauta de Evaluación Académica**
> La corrección del trabajo práctico priorizará la calidad y cobertura de la suite de pruebas unitarias creadas bajo TDD. Una interfaz visual atractiva sin una suite sólida de tests automáticos no alcanzará el estándar de aprobación.